

Data Analyst SQL Practice

Advanced Practice Set — Series 21 to 32

12 topics × 10 questions = 120 questions. Questions only; no solutions.

21. CASE WHEN — Conditional Logic

Functions / Concepts: CASE, WHEN, THEN, ELSE, END

1. Display each product name and classify its price: $\geq 50000 \rightarrow$ 'Expensive'; $\geq 10000 \rightarrow$ 'Medium'; $< 10000 \rightarrow$ 'Affordable'.
2. Display product name and classify stock: $\geq 50 \rightarrow$ 'High Stock'; $\geq 20 \rightarrow$ 'Medium Stock'; $< 20 \rightarrow$ 'Low Stock'.
3. Display customer name and classify their city: Delhi/Mumbai $\rightarrow$ 'Major City'; all other cities $\rightarrow$ 'Other City'.
4. Display order ID and classify order status: Delivered $\rightarrow$ 'Completed'; Cancelled $\rightarrow$ 'Failed'; everything else $\rightarrow$ 'Pending'.
5. Display product name, price, and a discount category: $\geq 50000 \rightarrow$ '20% Discount'; $\geq 10000 \rightarrow$ '10% Discount'; otherwise $\rightarrow$ '5% Discount'.
6. Display customer name and classify customer ID: $ID \leq 5 \rightarrow$ 'Group A'; $ID > 5 \rightarrow$ 'Group B'.
7. Display product name and stock status: stock = 0 $\rightarrow$ 'Out of Stock'; stock $\leq 10 \rightarrow$ 'Critical'; stock $\leq 30 \rightarrow$ 'Low'; otherwise $\rightarrow$ 'Available'.
8. Display order ID and classify order-item value using quantity * unit_price: $\geq 50000 \rightarrow$ 'High Value'; $\geq 10000 \rightarrow$ 'Medium Value'; otherwise $\rightarrow$ 'Low Value'.
9. Display customer name and classify customers based on number of orders: more than 3 $\rightarrow$ 'Frequent'; 2–3 $\rightarrow$ 'Regular'; 1 $\rightarrow$ 'New'.
10. Display each product and classify whether it needs restocking: stock < 10 AND price $> 10000 \rightarrow$ 'Urgent Restock'; stock $< 10 \rightarrow$ 'Restock'; otherwise $\rightarrow$ 'No Action'.

22. NULL Handling

Functions / Concepts: IS NULL, IS NOT NULL, COALESCE(), NULLIF()

1. Find customers whose city is NULL.
2. Find customers whose city is NOT NULL.
3. Find customers who do not have an email address.
4. Display customer name and replace NULL city with 'Unknown'.
5. Display customer name and replace NULL email with 'Not Provided'.
6. Display customer address and replace NULL state with 'Unknown'.
7. Find products where stock_quantity is NULL.
8. Display product name and replace NULL stock_quantity with 0.
9. Find orders where order_status is NULL.
10. Display customer name and city, replacing NULL city with 'Not Available'.

23. String Functions

Functions / Concepts: CONCAT(), LOWER(), UPPER(), TRIM(), LENGTH(), SUBSTRING(), LEFT(), RIGHT(), REPLACE()

1. Display customer first name and last name as one full_name.

2. Display all customer names in uppercase.
3. Display all customer emails in lowercase.
4. Find the length of each customer's first name.
5. Display the first 3 characters of every customer's first name.
6. Display the last 3 characters of every customer's last name.
7. Display the first 5 characters of every product name.
8. Replace 'Phone' with 'Mobile' in product names.
9. Display customer names after removing leading/trailing spaces using TRIM().
10. Create an email label using first_name + ' - ' + email. Example: Rahul - rahul@gmail.com.

24. Date Functions

Functions / Concepts: YEAR(), MONTH(), DAY(), DATE(), EXTRACT(), DATEDIFF(), DATE_ADD(), DATE_SUB()

1. Display each order and extract the year from order_date.
2. Display each order and extract the month from order_date.
3. Display each order and extract the day from order_date.
4. Count how many orders were placed in each year.
5. Count how many orders were placed in each month.
6. Find customers who registered after 2025-03-01.
7. Find the number of days between customer registration_date and '2025-06-30'.
8. Find orders placed within the last 30 days from '2025-06-30'.
9. Display the order date and the date 7 days after the order.
10. Find the number of days between the earliest and latest order.

25. Advanced Aggregation

Functions / Concepts: COUNT(), SUM(), AVG(), MIN(), MAX(), COUNT(DISTINCT)

1. Find the total number of unique customers who placed orders.
2. Find the total revenue generated from all orders.
3. Find the average order value.
4. Find the highest order value.
5. Find the lowest order value.
6. Find the total quantity of products sold.
7. Find the average quantity per order.
8. Find the total revenue generated by each customer.
9. Find the average order value for each customer.
10. Find the percentage of orders that are Delivered.

26. Subqueries

Functions / Concepts: Scalar Subquery, Subquery in WHERE, Subquery in FROM, Correlated Subquery, IN, EXISTS

1. Find products whose price is greater than the average product price.
2. Find products whose price is equal to the highest product price.
3. Find products whose price is less than the average product price.
4. Find customers who have placed at least one order.
5. Find customers who have never placed an order.
6. Find products that have been ordered at least once.
7. Find products that have never been ordered.
8. Find the second highest product price.
9. Find customers whose number of orders is greater than the average number of orders per customer.
10. Find the customer who has spent the most money.

27. CTEs — Common Table Expressions

Functions / Concepts: WITH, CTE, Multiple CTEs

1. Create a CTE containing all products with price > 10000.
2. Create a CTE containing total quantity sold for each product.
3. Using a CTE, find products whose total quantity sold is greater than 2.
4. Using a CTE, calculate the total revenue for each product.
5. Using a CTE, find the top 5 products by revenue.
6. Using a CTE, calculate total spending for each customer.
7. Using a CTE, find customers who spent more than 50000.
8. Create a CTE for order totals and find the average order value.
9. Create a CTE for category revenue and find the highest-revenue category.
10. Create two CTEs: total revenue per customer and total orders per customer; then display both values for each customer.

28. Window Functions — Ranking

Functions / Concepts: ROW_NUMBER(), RANK(), DENSE_RANK(), OVER(), PARTITION BY, ORDER BY

1. Rank all products by price from highest to lowest using RANK().
2. Rank all products by price from lowest to highest.
3. Assign a row number to every product ordered by price descending.
4. Rank products within each category based on price.
5. Find the top 3 most expensive products in each category.
6. Find the cheapest product in each category using ROW_NUMBER().
7. Rank customers based on their total spending.
8. Rank customers based on number of orders.
9. Find the top 3 customers by revenue.
10. Assign a ranking to orders based on order value, highest order value first.

29. Window Functions — Analytical

Functions / Concepts: SUM() OVER(), AVG() OVER(), COUNT() OVER(), LAG(), LEAD(), PARTITION BY, ORDER BY

1. Calculate a running total of order revenue ordered by order_date.
2. Calculate a running total of quantity sold ordered by order_id.
3. Calculate the average product price across all products using a window function.
4. Display each product's price and the average price of its category.
5. Display each product's price and the difference between its price and its category's average price.
6. Display each order and the previous order's date using LAG().
7. Display each order and the next order's date using LEAD().
8. Find the difference in days between an order and the previous order.
9. Display each customer's order and their previous order date.
10. Calculate each product's percentage contribution to total revenue.

30. Set Operators

Functions / Concepts: UNION, UNION ALL, INTERSECT, EXCEPT

1. Create a list containing all customer cities and all address cities using UNION.
2. Create a list containing all customer cities and all address cities using UNION ALL.
3. Find cities that exist in both customers and addresses using INTERSECT.
4. Find cities that exist in customers but not in addresses using EXCEPT.
5. Combine customer first names and product names into one result using UNION.
6. Combine customer emails from two different filtered queries using UNION.
7. Combine products priced above 50000 and products with stock below 10 using UNION.
8. Combine all Delivered and Shipped order IDs using UNION.
9. Combine two result sets of customer IDs using UNION ALL.
10. Find customer IDs who appear in both two filtered order sets using INTERSECT.

31. Advanced JOINS

Functions / Concepts: INNER JOIN, LEFT JOIN, RIGHT JOIN, SELF JOIN, CROSS JOIN, Multiple JOIN, Anti JOIN

1. Find customers who have placed at least one order.
2. Find customers who have never placed an order.
3. Find products that have never been ordered.
4. Find categories that have no products.
5. Find customers who have both an order and an address.
6. Find customers whose city is different from their address city.
7. Display every customer and their total number of orders, including customers with zero orders.
8. Display every product and its total quantity sold, including products that have never been ordered.
9. Find categories and their total revenue, including categories with zero revenue.
10. Display every customer with their total spending, including customers who have never placed an order.

32. Real Data Analyst SQL — Mixed Problems

Functions / Concepts: JOIN, WHERE, GROUP BY, HAVING, CASE, CTE, Subquery, Window Functions, ORDER BY, LIMIT

1. Find the top 5 customers by total revenue.
2. Find the top 3 products by revenue in each category.
3. Find customers who have placed more than 2 orders and spent more than $\geq 10,000$.
4. Find products that have high stock (> 30) but total quantity sold is less than 5.
5. Find the category with the highest average product price.
6. Find the month that generated the highest total revenue.
7. Find customers who placed their first order within 30 days of registration.
8. Find the percentage of total revenue generated by each category.
9. Find the customer with the highest average order value.
10. Find the second-highest revenue-generating product.